

Equation Automata

Without using any math functions,
outputting a trigonometric equation

by

GOKTUG ISLAMOGLU

goktugislamoglu@gmail.com

GOKTUG ISLAMOGLU from Freiburg, Germany



Data Steward / Data Custodian
B.Sc Istanbul Technical University
M.Sc Karadeniz Technical University

INFORMS 2018 Session Chair
ITS SUMMIT 2019 Presentation
NECSI ICCS 2020 Presentation
NERCSS 2021 Presentation
ALIFE ERA 2025 Presentation

Model: Equation Automaton¹

Runs upon: PyCX 0.3 Realtime Visualization Template

Requires PyCX simulator to run

PyCX available to download from:
<https://github.com/hsayama/PyCX>

1. <https://github.com/goektug/Equation-Automata/>

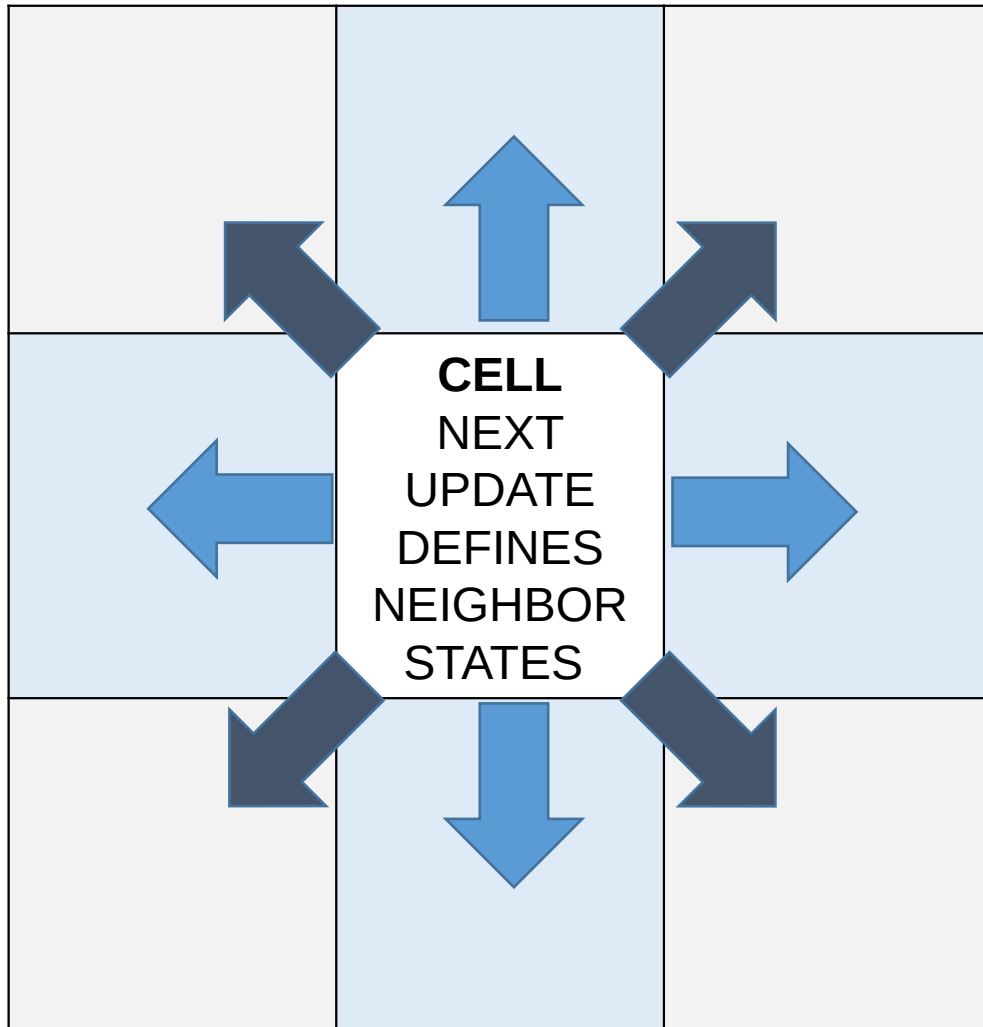
Conventional Cellular Automaton Update

	CELL NEXT UPDATE DEFINED BY NEIGHBOR STATES	

A cellular automaton is a search function around a cell.

Update rule is the determination of a cell's state based on the states of its neighboring cells following a rule.

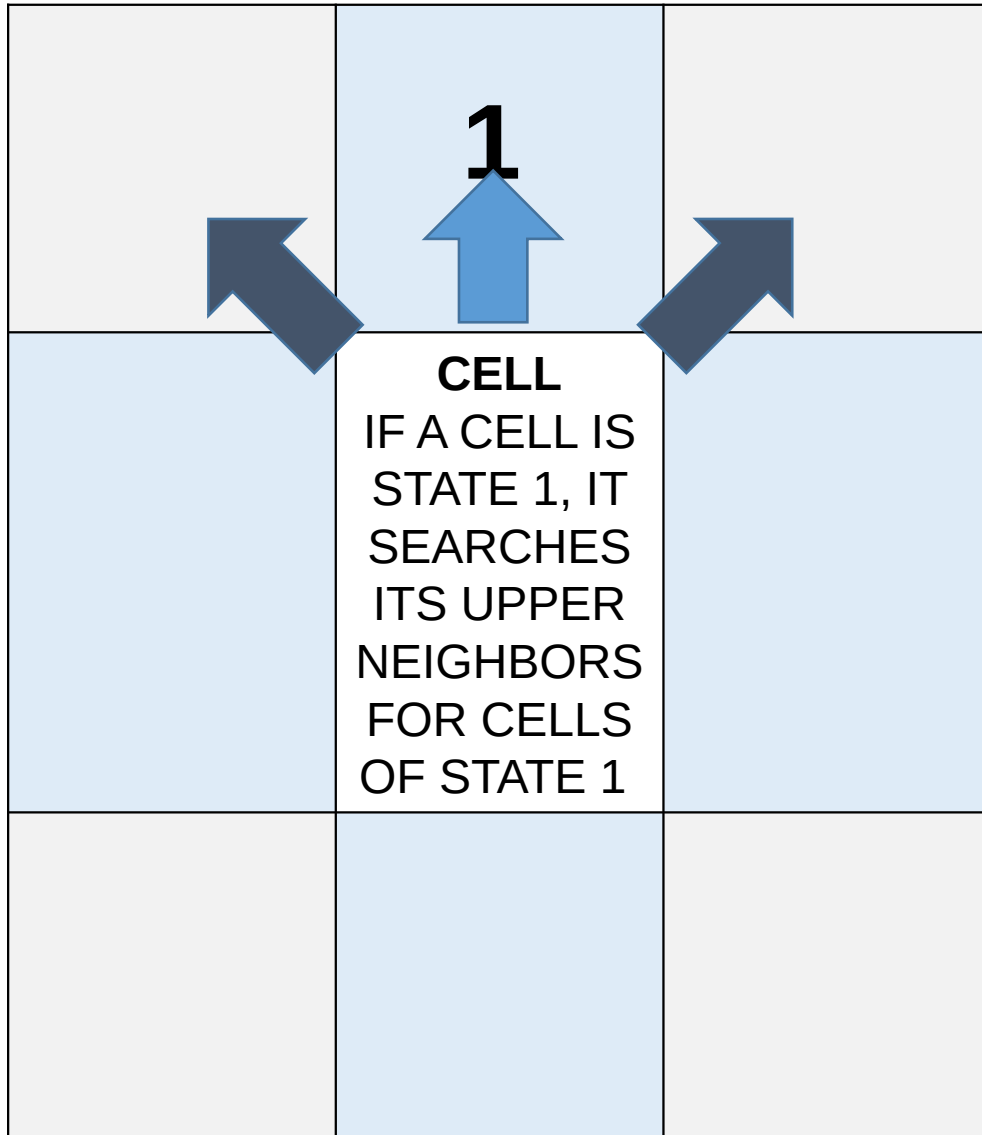
1st Rule: Inverse Cell Update



To achieve driving motion, update of a cell is reversed. Cell's state dictates its neighborhood.

```
elif c[x, y] == 1:  
    array1.append(c[x, y])  
    for z in range(-1, 2):  
        # block generation from randomly distributed points
```

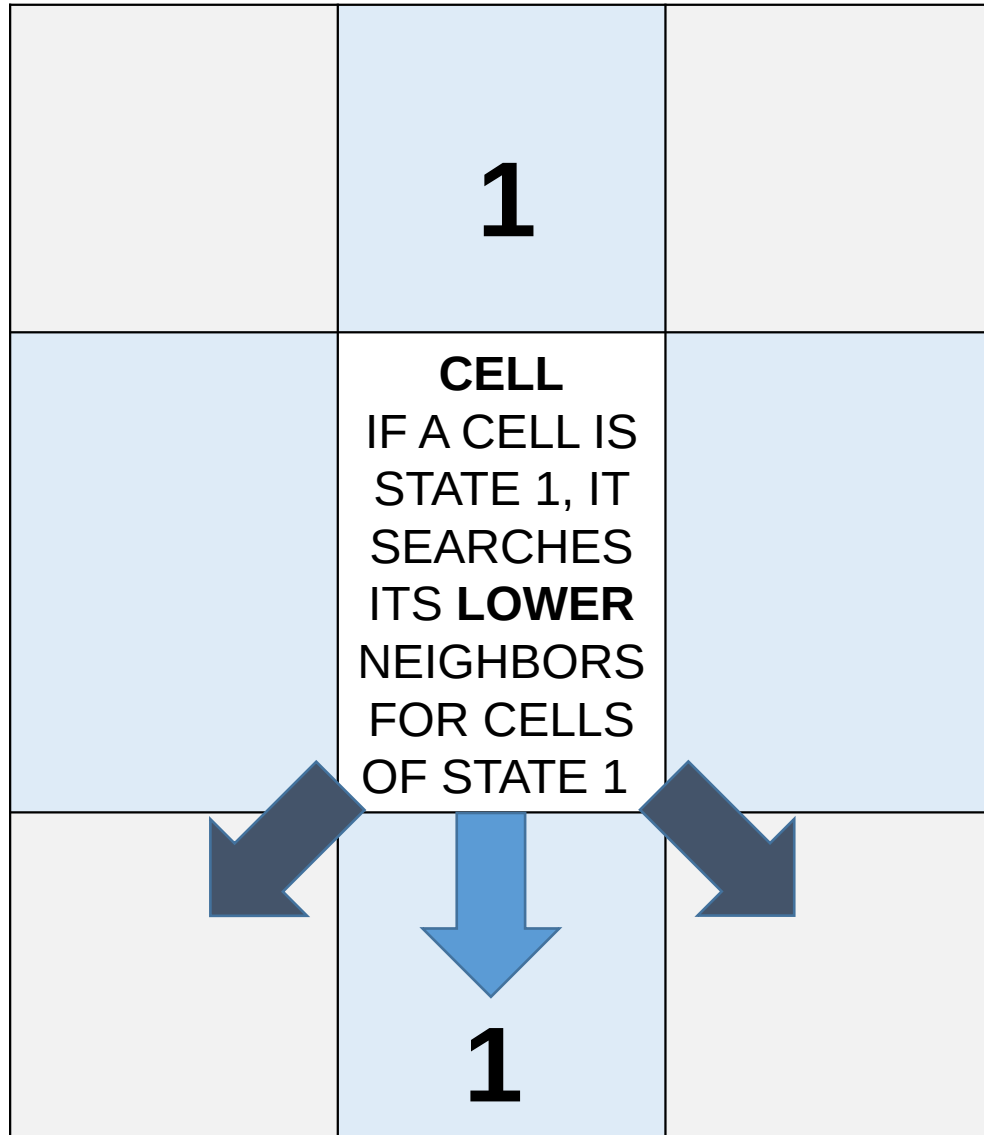
Updating upper neighboring cells



If a cell in the upper neighborhood is found to be state 1, the middle cell is updated to 1.

```
elif c[x, y] == 1:  
    array1.append(c[x, y])  
    for z in range(-1, 2):  
        # block generation from randomly distributed points  
  
        #neighbor updating from cell(x,y)  
        m = number_of_upper_neighbors(x, y)  
        if m == 1:  
            nc[x, (y + 1) % L] = 1
```

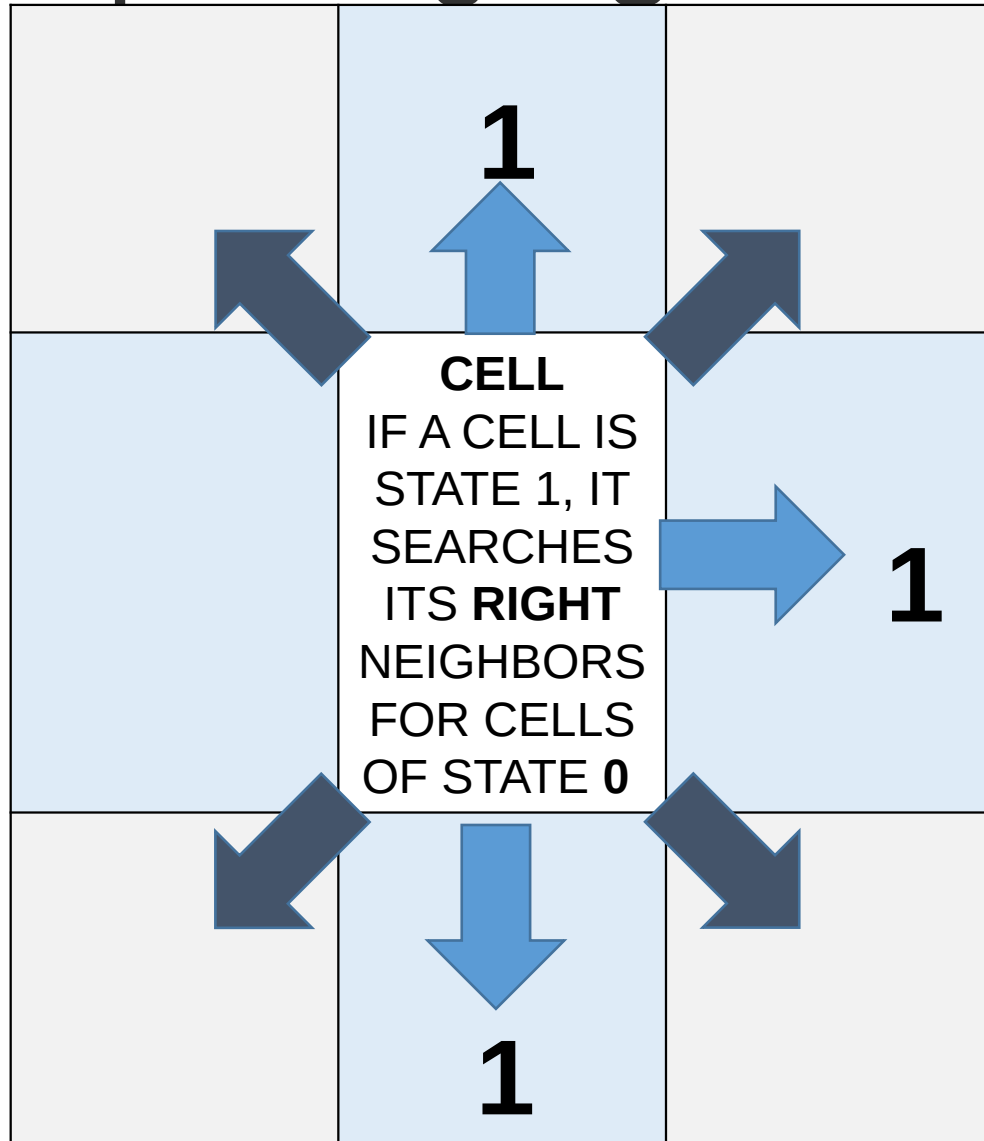
Updating **lower** neighboring cells



If a cell in the **lower** neighborhood is found to be state 1, the middle cell is updated to 1.

```
n = number_of_lower_neighbors(x, y)
if n == 1:
    nc[x, (y - 1) % L] = 1
```

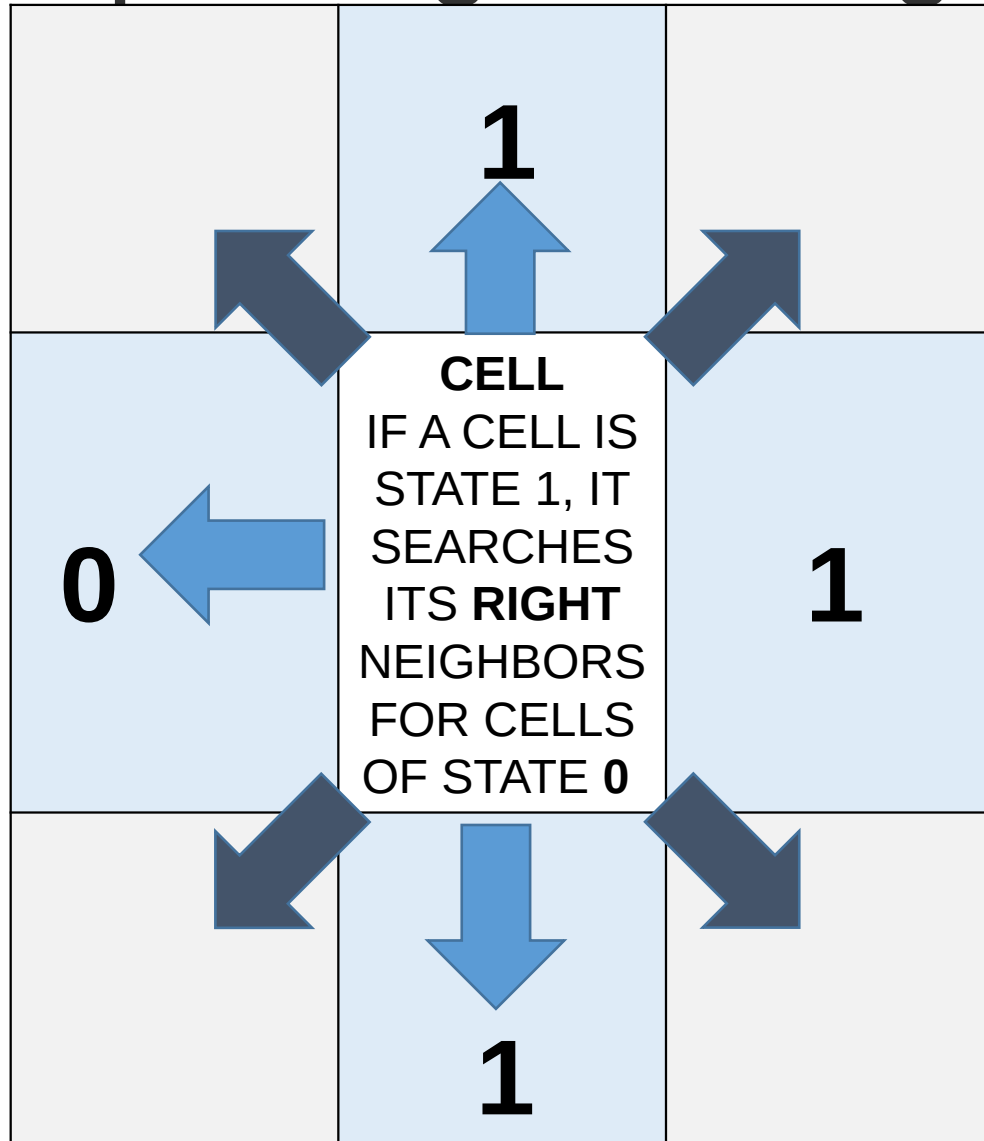
Updating **right** neighboring cells



If the cells in the **right** neighborhood is found to be state **0**, and the upper and lower neighbors are **less than or equal to one**, the middle cell is updated to 1.

```
k = number_of_right_neighbors(x, y)
if k == 0 and (m <= 1 or n <= 1):
    nc[(x + 1) % L, (y + z) % L] = 1
```


Updating **left** neighboring cells



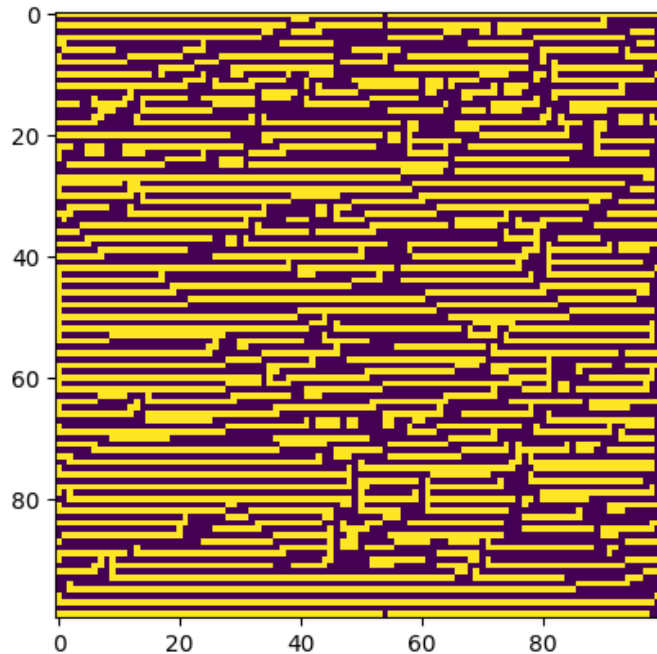
If the cells in the **left** neighborhood is found to be state **0**, and the upper and lower neighbors are **more than one**, the middle cell is updated to 0.

```
l = number_of_left_neighbors(x, y)
if l == 1 and (m > 1 or n > 1):
    nc[(x - 1) % L, (y + z) % L] = 0
```

Idea

Critical behavior by extending cells of an automaton uniaxially

This is called the “driving motion”, aimed at maximizing cell count

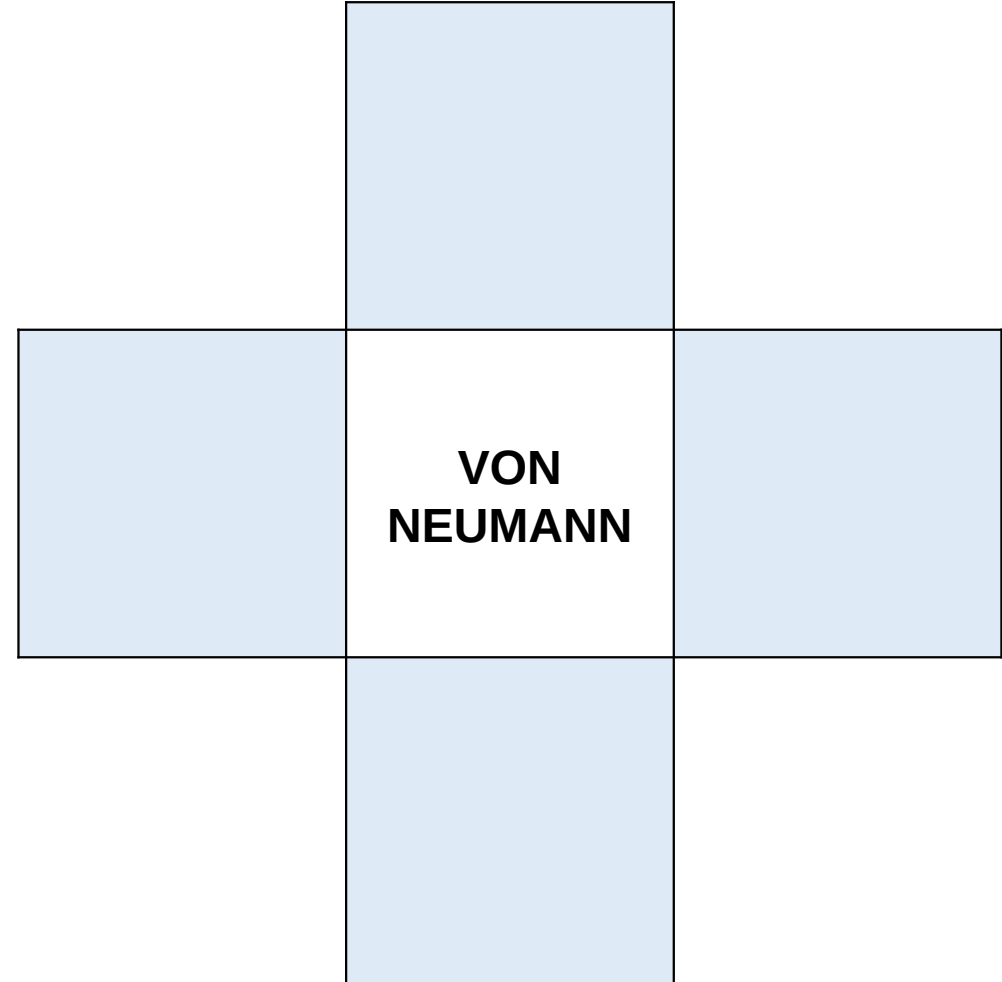
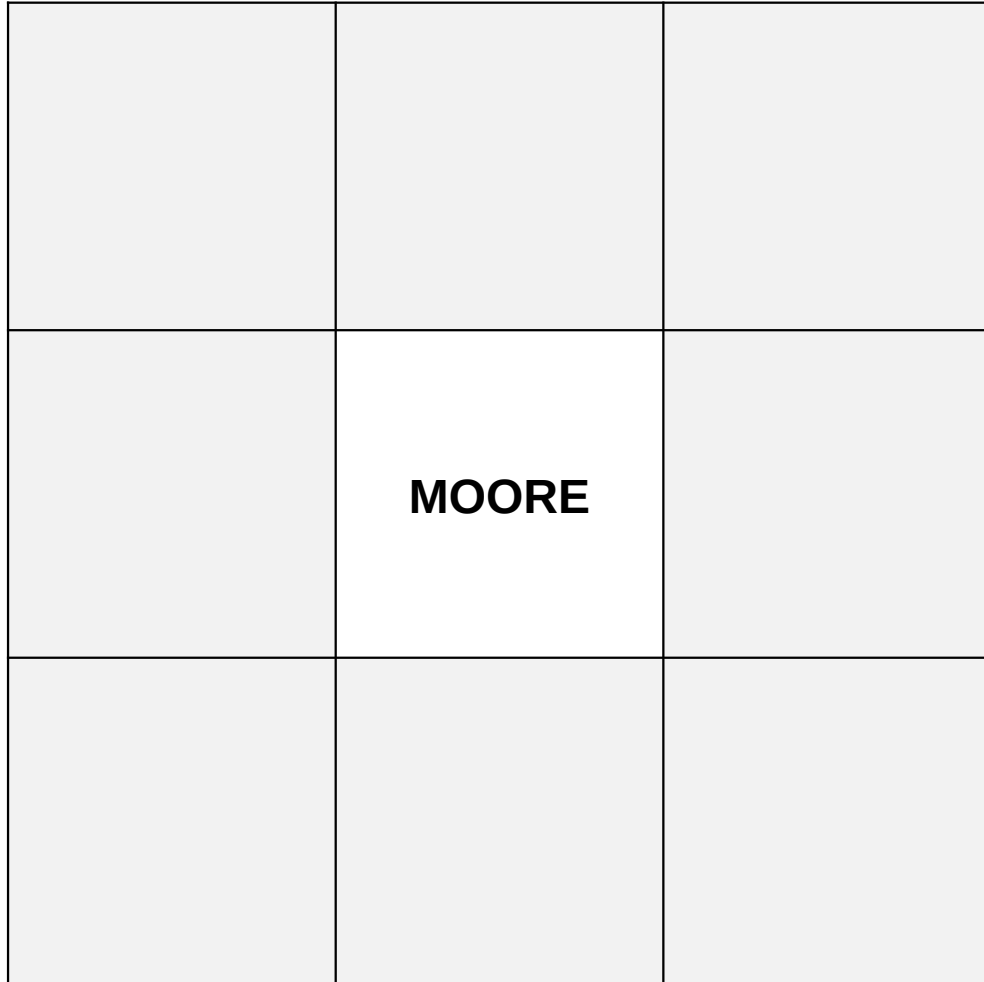


Left: Maximum cell count

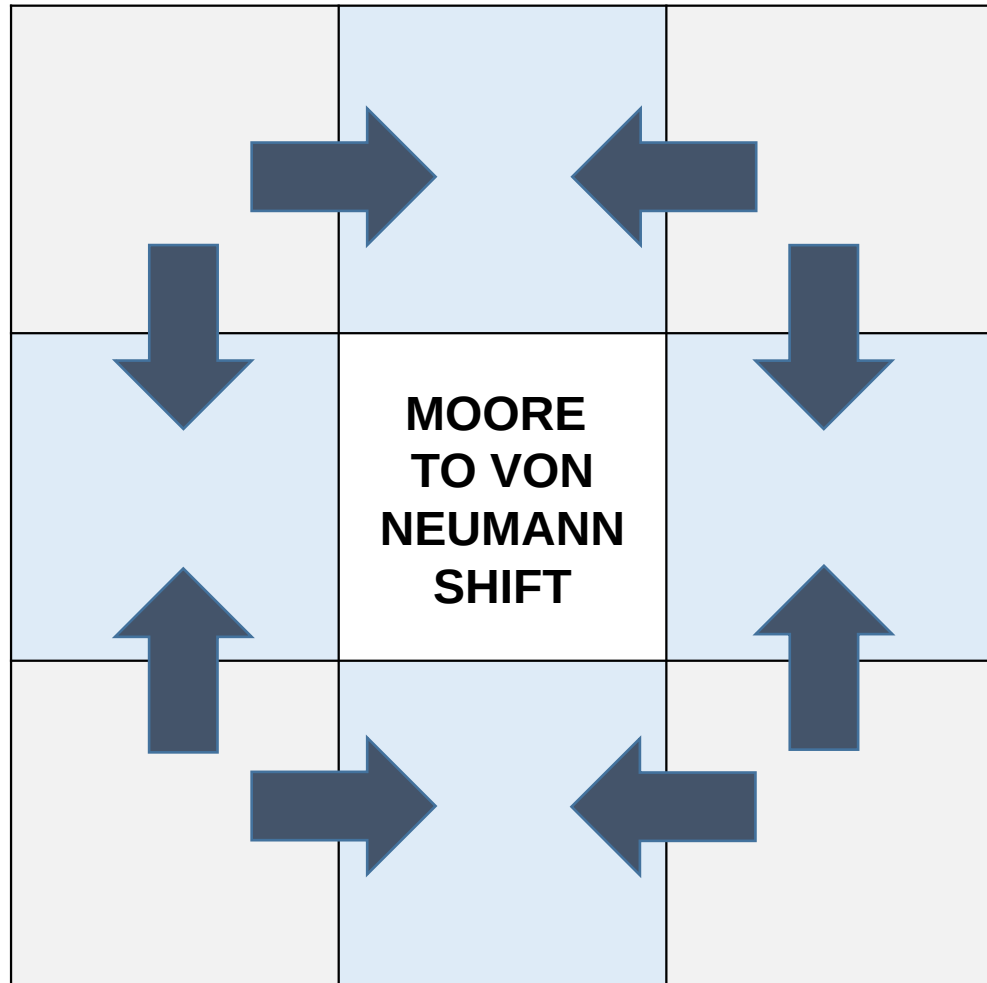
Ferromagnetic in horizontal

Antiferromagnetic in lateral

Moore vs Von Neumann Neighborhoods



2nd Rule: Moore to Von Neumann Shift



To achieve criticality, competition between Moore cells and von Neumann cells are needed.

```
g = number_of_Moore_neighbors(x, y) #CA tuning
if c[x, y] == 0:
    nc[x, y] = 0 if g <= 6 else 1
    array0.append(c[x, y])

h = number_of_Neumann_neighbors(x, y) #CA tuning
if h >= 1:
    nc[x, y] = 1 if g <= 6 else 0
```

2nd Rule: Neighborhood Configuration

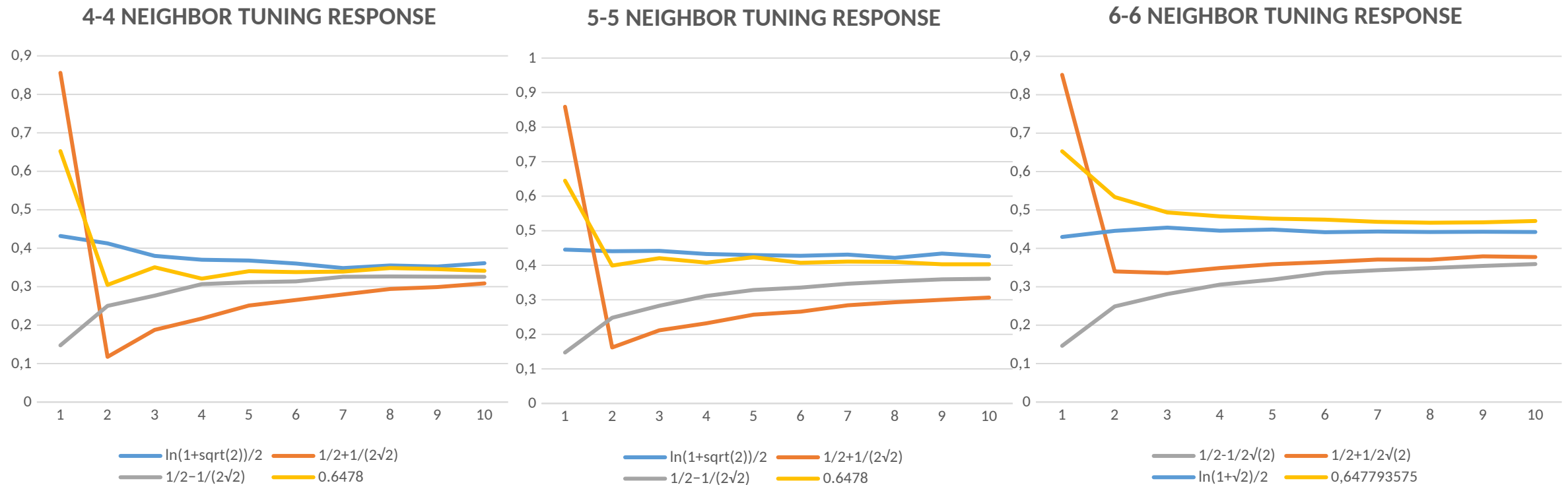
1	1	1
0	TUNING THE NEIGHBOR NUMBERS	0
1	1	1

Based on cell's value, Moore and von Neumann neighborhoods are tuned. Critical values are 6 for both.

```
g = number_of_Moore_neighbors(x, y) #CA tuning
if c[x, y] == 0:
    nc[x, y] = 0 if g <= 6 else 1
    array0.append(c[x, y])

h = number_of_Neumann_neighbors(x, y) #CA tuning
if h >= 1:
    nc[x, y] = 1 if g <= 6 else 0
```

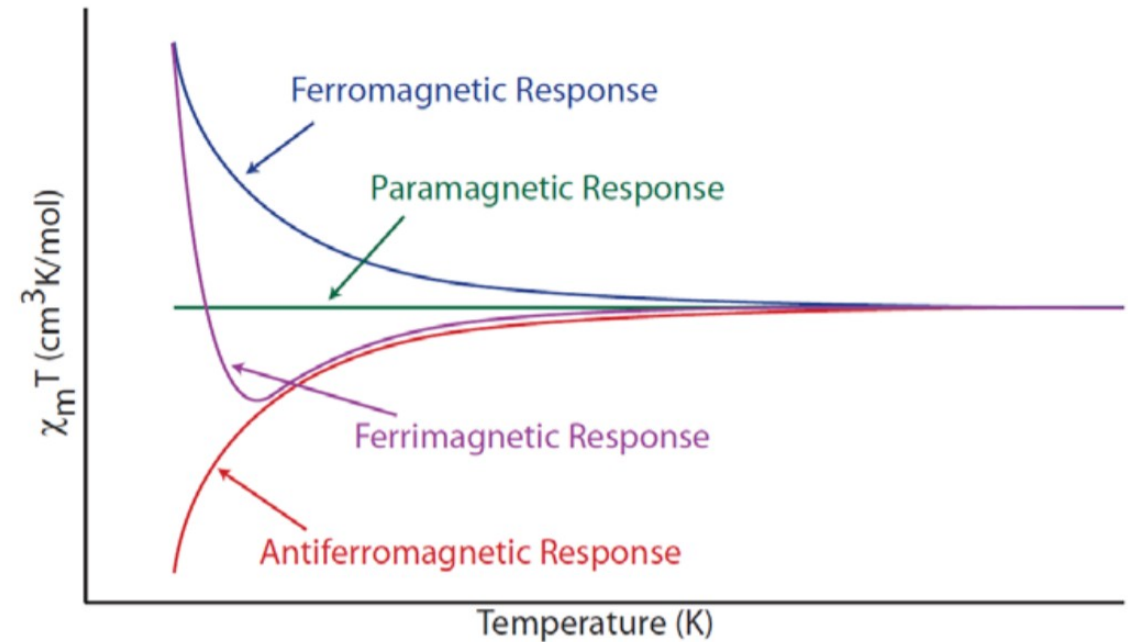
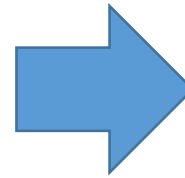
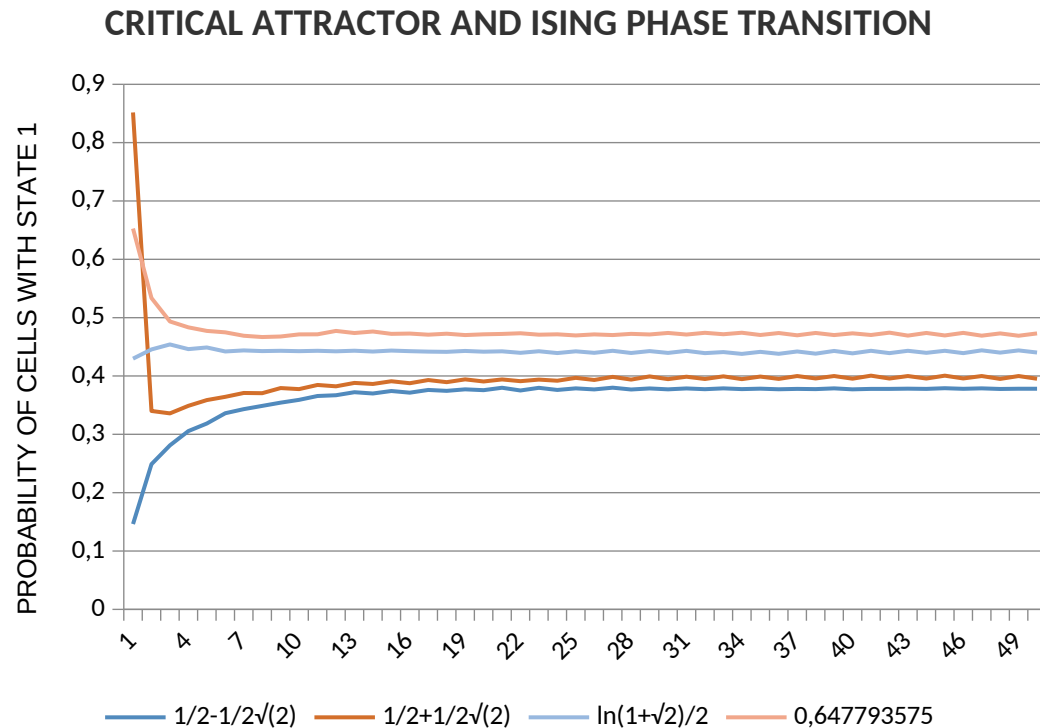
Neighbor Tuning for Ising Criticality



Tuning for **4** and **5** neighbors has **Inverse Ising critical temperature** as the highest state 1 cell count.

For **6** neighbors however, there is another maximum susceptibility, corresponding to ferromagnetism.

Ising Model criticality



2D Ising square lattice model's **critical inverse temperature** is the **paramagnetic response**.

Upper criticality is weak ferromagnetism (**ferrimagnetism**).

Lower criticality is **antiferromagnetic** behavior.

3rd Rule: Critical Cellular Automata

$$g/8 > (1 - p) * p$$

$$p^2 - p + 1/8 = 0$$

$$p = 1/2 \pm 1/(2\sqrt{2})$$

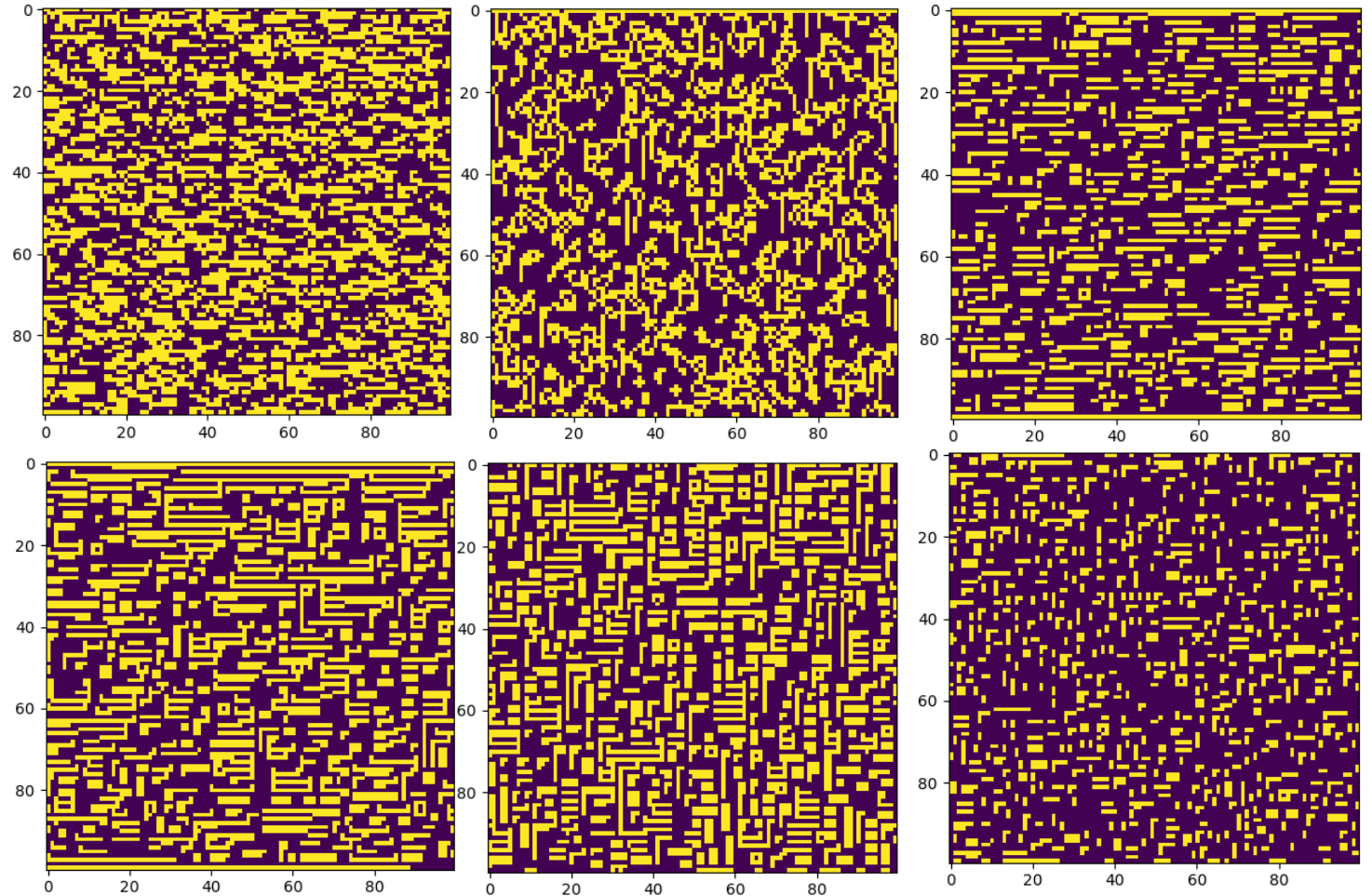
Left: 2D criticality condition broken down to its roots by assigning **1** to Moore neighborhood count g and creating an equation.

```
if g / 8 > (1 - p) * p: # coupling function
    nc[(x + 1) % L, y] = 1
elif g / 8 < (1 - p) * p:
    nc[(x - 1) % L, y] = 1
else:
    nc[x, y] = 1
```

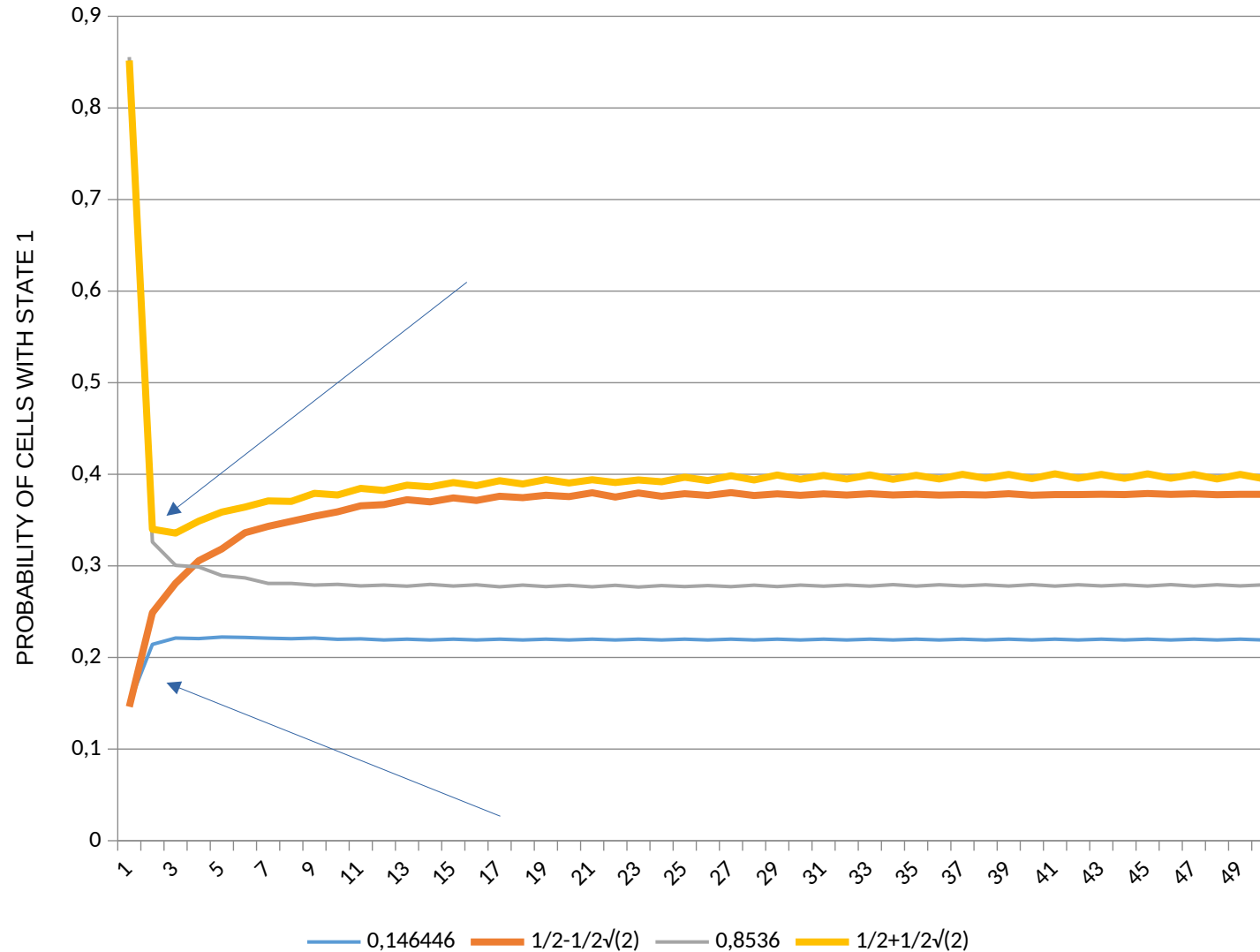

Evolution of Criticality

CLOCKWISE FROM TOP:

1. Neighbor update without criticality
2. Criticality without neighbor update
3. Uncritical state above $\frac{1}{2} \pm 1/(2\sqrt{2})$
4. Uncritical state below $\frac{1}{2} \pm 1/(2\sqrt{2})$
5. Critical state with neighbor update, $\frac{1}{2} - 1/(2\sqrt{2})$
6. Critical state with neighbor update, $\frac{1}{2} + 1/(2\sqrt{2})$



First-Order Phase Transition



Trigonometric Expression of Criticality

$$\bullet \cos^2\left(\pi/8\right) = \frac{1}{2} + \frac{1}{2\sqrt{2}}$$

$$\sin^2\left(\pi/8\right) = \frac{1}{2} - \frac{1}{2\sqrt{2}}$$

$$\cos^2\left(\pi/8\right) - \sin^2\left(\pi/8\right) = \frac{1}{\sqrt{2}} = 2 \sin\left(\pi/8\right) \cos\left(\pi/8\right)$$

General Expression:

$$a \cos^2 x - b \sin^2 x = \sin x \cos x$$

Trigonometric Expression of Criticality

divided by $\cos^2(x)$ on both sides:

$$\therefore b \tan^2(x) + \tan(x) - a = 0$$

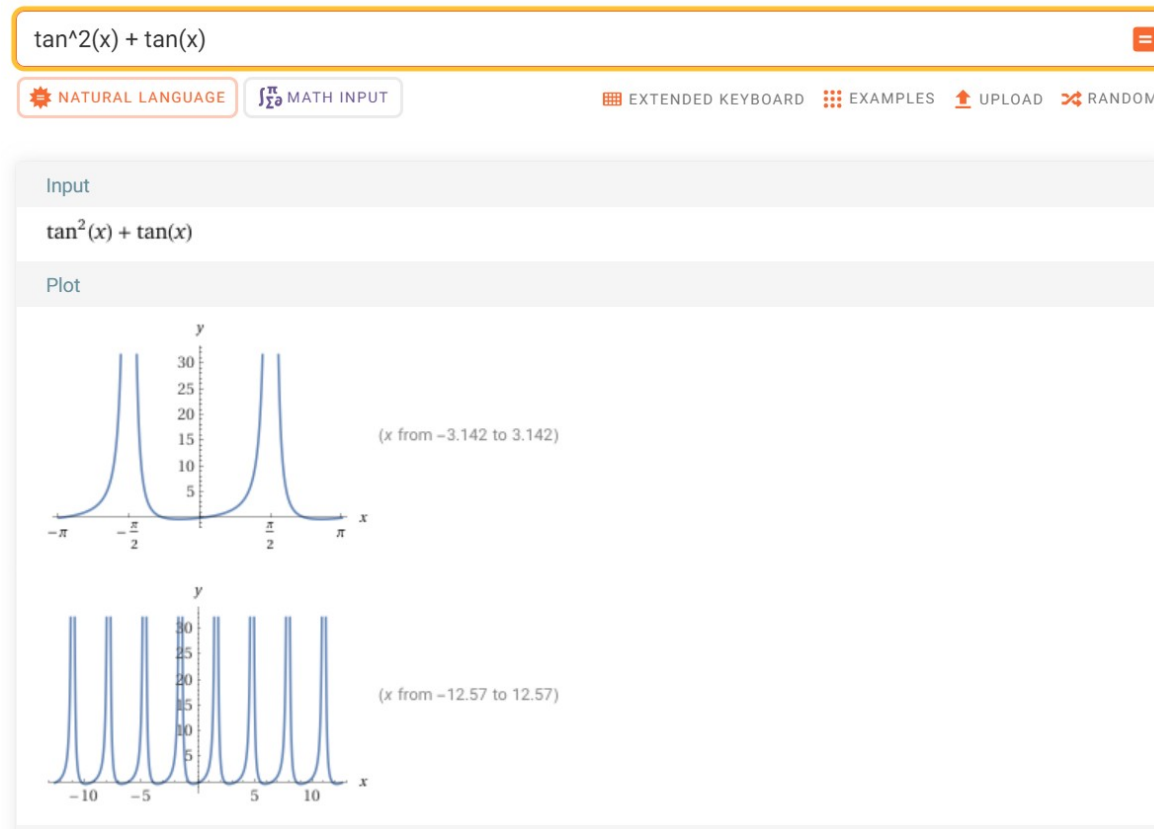
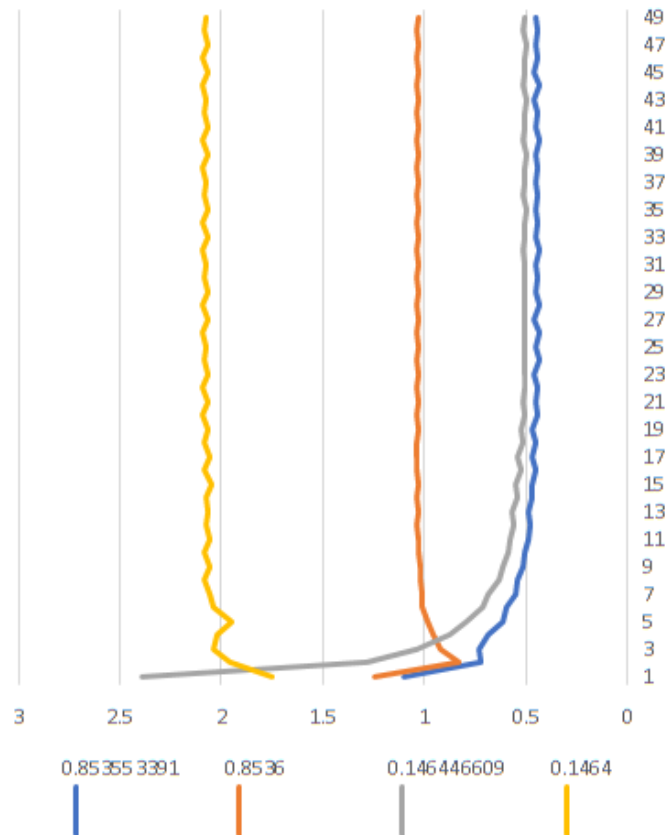
which corresponds to the code that draws the tangent equation.

Tangent Graph Output of CA

Critical and uncritical values are arms of the tangent graph.



$\tan^2 x$
+ $\tan x$
Graph



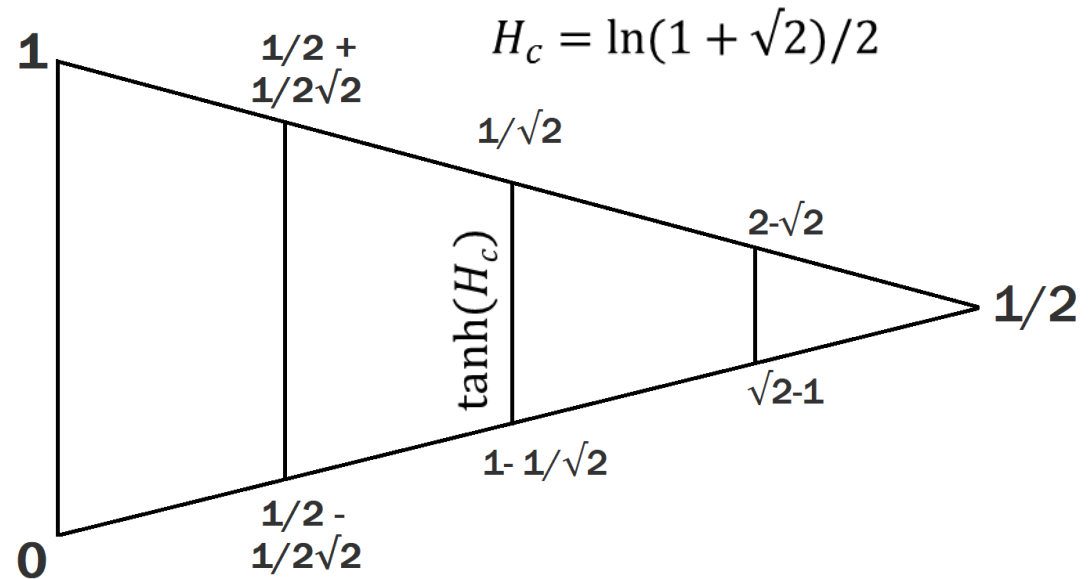
Ising Model Correspondence Visualized

Stereographic projection of 2D rectangular
Ising model to 1D linear Ising chain

$$s = \tan(1/2 * \pi/4)$$

$$s = \tanh(1/2 * \ln(1+\sqrt{2}))$$

$$\tan(\pi/8) = \tanh(\ln(1+\sqrt{2})/2)$$



Open Question: How to Extend Deep into Ising Model?

init p	x output	$\tan^2 x + \tan x$	$\tanh$	$\tanh+1$	$(\tanh+1)/2$
0.7071	0.3198	0.4409	0.4144	1.4144	0.7072

